

LAPORAN PRAKTIKUM
MATAKULIAH PENGOLAHAN CITRA DAN VISI KOMPUTER



Disusun Oleh :

Siti Nikmatus Sholihah

NIM. 244107020014

PROGRAM STUDI SARJANA TERAPAN TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG

2026

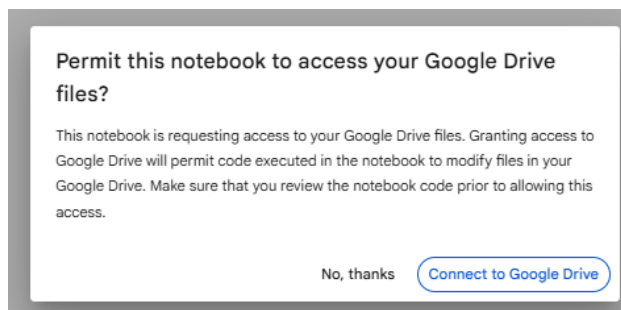
D1. Pengenalan Python: Pengolahan Citra Dasar dan memahami channel warna pada OpenCV dan konversinya

1. Membuat koneksi antara Colab dengan Google Drive agar bisa mengakses file di dalamnya.

Menggunakan file yang sama dengan praktikum pada D2, import folder yang ada di Drive Anda dengan cara sebagai berikut

```
[ ] from google.colab import drive
    drive.mount('/content/drive')
```

Setelah kode tersebut dijalankan, akan muncul pop up untuk terhubung ke Google Drive

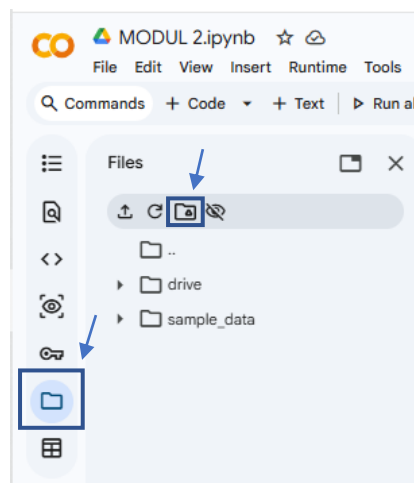


Jika berhasil, maka Google Colab sudah dapat mengakses folder gdrive Anda dengan keterangan output program “Mounted at /content/drive”.

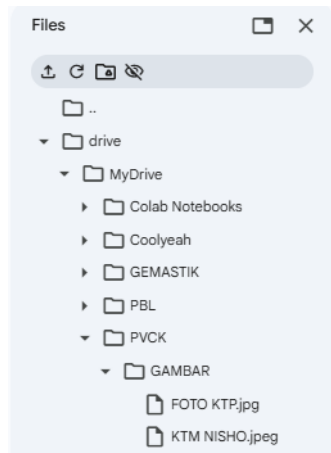
```
[1] ✓ 1m from google.colab import drive
      drive.mount('/content/drive')
      Mounted at /content/drive
```

Cara yang kedua dengan:

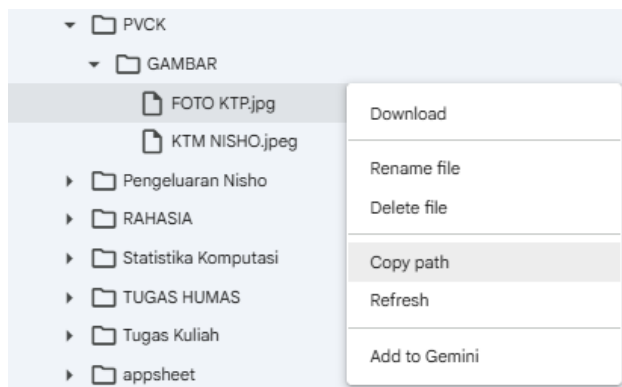
- a. mengklik tombol folder di sebelah kiri



- b. Pilih tombol gdrive Connect to Google Drive. Drive akan muncul dalam list folder:

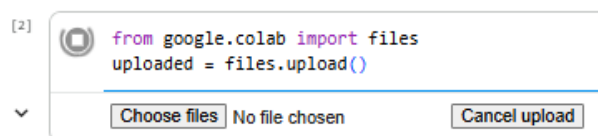


Pilih folder dan klik icon **titik 3** disebelah kanan file gambar yang akan digunakan. klik **copy path**, dan bisa digunakan pada code berikutnya



Kemudian kode program dapat dilanjutkan dengan membuka file Google Drive yang sudah ada. Contohnya pada kode program di bawah file image dengan nama **ktp.jpg** akan dibuka untuk diproses lebih lanjut.

Penjelasan diatas adalah cara mengakses gambar yang tersimpan di Google Drive (Cloud), namun jika posisi gambar tersimpan di computer local/laptop maka cara aksesnya bisa menggunakan fungsi **upload gambar**. Adapun caranya adalah sebagai berikut:



Jika **Choose Files** di klik, akan muncul direktori dari computer local. Setelah dipilih gambar yang akan ditampilkan atau diolah, akan muncul tampilan sebagai berikut yang menunjukkan gambar sudah tersimpan di google colab.

```
[2]
✓ 42s
from google.colab import files
uploaded = files.upload()
```

... Choose files FOTO KTP.jpg
FOTO KTP.jpg(image/jpeg) - 108655 bytes, last modified: 01/09/2026 - 100% done
Saving FOTO KTP.jpg to FOTO KTP.jpg

Untuk menampilkan gambar tersebut, maka perintahnya adalah sebagai berikut:

```
[3]
✓ 0s
import cv2
from google.colab.patches import cv2_imshow

img = cv2.imread("FOTO KTP.jpg")
cv2_imshow(img)

print("Resolusi gambar:", img.shape[1], "x", img.shape[0])
```

... 
Resolusi gambar: 736 x 465

2. Menampilkan Gambar

Untuk menampilkan Gambar, bisa menampilkan langsung (*Image Display*) salah satunya menggunakan library openCV dengan fungsi `cv2.imshow()` atau `cv2_imshow()` di Google Colab atau menampilkan dalam Plot (*Image Plotting*) menggunakan library Matplotlib (`plt.imshow()`). Berikut ini cara menampilkan gambar dalam Plot supaya bisa diketahui ukuran lebar dan Panjang dari Image:

```
[7]
✓ 0s
import matplotlib.pyplot as plt
img = cv2.imread("FOTO KTP.jpg")
plt.imshow(img)
```

Hasil nya berupa output gambar yang sudah di plot dengan matplotlib dapat digunakan untuk mengetahui ukuran panjang dan lebar dari gambar tersebut. Perhatikan format warna yang ditampilkan, **warna dengan format BGR.**



OpenCV membaca image dan menyimpan dalam **channel warna BGR** (Blue Green Red). Sehingga saat ditampilkan pada plotter juga akan ditampilkan dalam format warna BGR. Jika menginginkan **format warna RGB** dapat dilakukan dengan beberapa cara:

```
[11] img = cv2.imread("FOTO KTP.jpg")
      img2 = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
      plt.imshow(img2)
```

Atau langsung digabung:

```
[10] img = cv2.imread("FOTO KTP.jpg")
      plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
      plt.show()
```

Perhatikan ada 2 fungsi plt: plt.imshow (img2) dan plt.show(). Perbedaan keduanya:

- plt.imshow(img2) → digunakan untuk memasukkan gambar ke dalam plot.
- plt.show() → hanya dipanggil **tanpa argumen** untuk menampilkan plot.



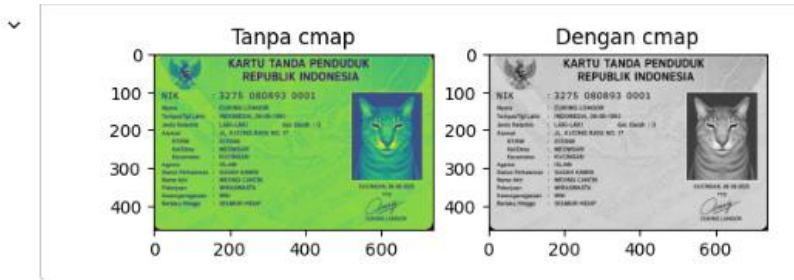
3. Menampilkan citra Grayscale:

```
[12]
✓ Os
img_gray = cv2.imread("FOTO KTP.jpg", cv2.IMREAD_GRAYSCALE)

# Tanpa cmap
plt.subplot(1,2,1)
plt.imshow(img_gray)
plt.title("Tanpa cmap")

# Dengan cmap
plt.subplot(1,2,2)
plt.imshow(img_gray, cmap="gray")
plt.title("Dengan cmap")

plt.show()
```



Jika ingin mengganti warna yang lain, tinggal mengganti cmap sesuai keinginan, contohnya untuk ditampilkan colormap dengan warna 'magma'

```
[13]
✓ Os
img_gray = cv2.imread("FOTO KTP.jpg", cv2.IMREAD_GRAYSCALE)

# Coba Magma
plt.subplot(1,2,2)
plt.imshow(img_gray, cmap="magma")
plt.title("Dengan cmap")

plt.show()
```



4. Melakukan resizing

Untuk mengubah ukuran citra dari ukuran 694 x 458 menjadi ukuran 347 x 229, maka harus menggunakan fungsi resize dari library OpenCV:

```
[14]
✓ Os
imgRGB = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
imgRGBsmall = cv2.resize(imgRGB, (400, 250))
plt.imshow(imgRGBsmall)
```

Ukuran baru bisa dilihat dari angka pada plotter:



5. Melakukan Flipping

Untuk menampilkan Citra RGB dalam posisi terbalik menggunakan fungsi **flip** dari library opencv:

```
[16]  imgFlip = cv2.flip(imgRGBsmall, 1)  
plt.imshow(imgFlip)
```

Output yang ditampilkan gambar terbalik:



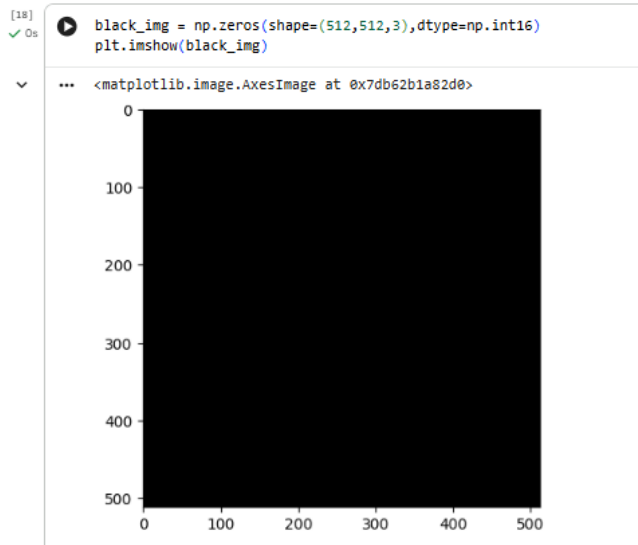
Sedangkan untuk memperbesar ukuran canvas yang lebih besar dengan posisi gambar terbalik:

```
[17]  imgFlip=cv2.flip(imgRGBsmall, 1)  
  
fig=plt.figure(figsize=(10,10))  
ax=fig.add_subplot(111)  
ax.imshow(imgFlip)
```

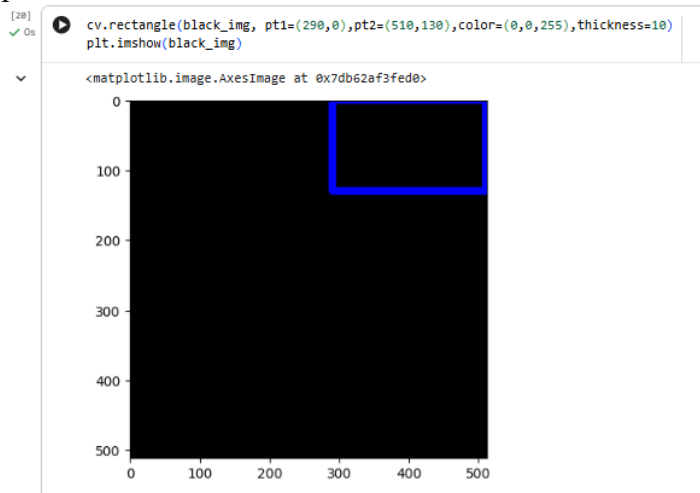
Hasilnya dengan ukuran canvas yang lebih besar:



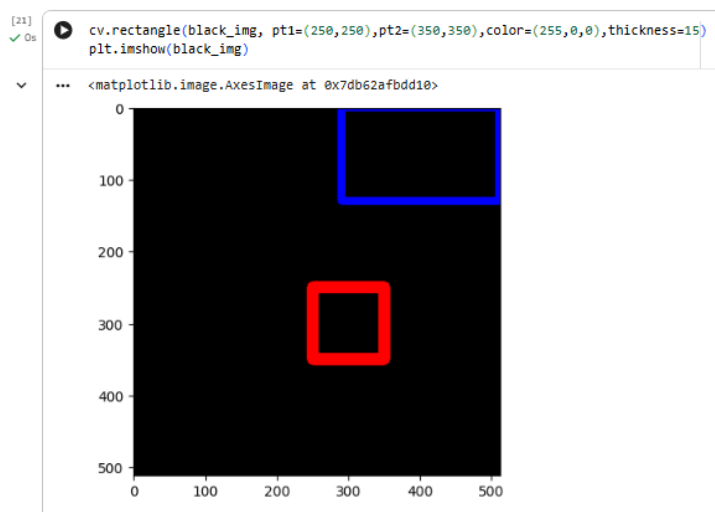
6. Membuat bentuk Geometri 2D dari OpenCV. Diawali dengan pembuatan black image dengan tipe data int16.



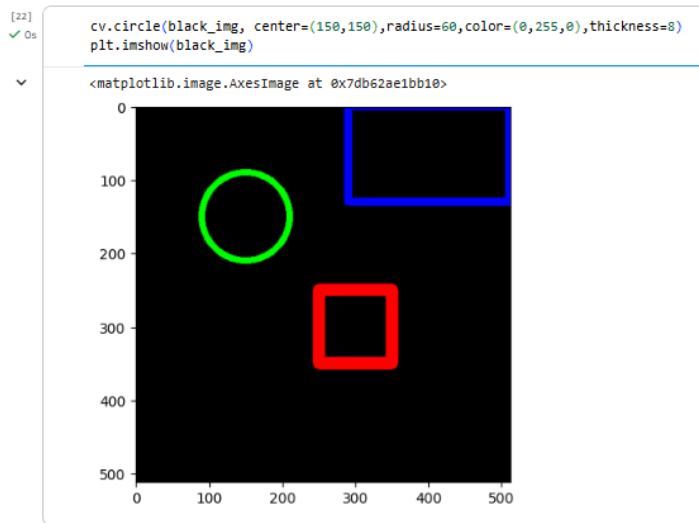
Kemudian menambahkan bentuk persegi panjang sesuai koordinat pt1 dan pt2



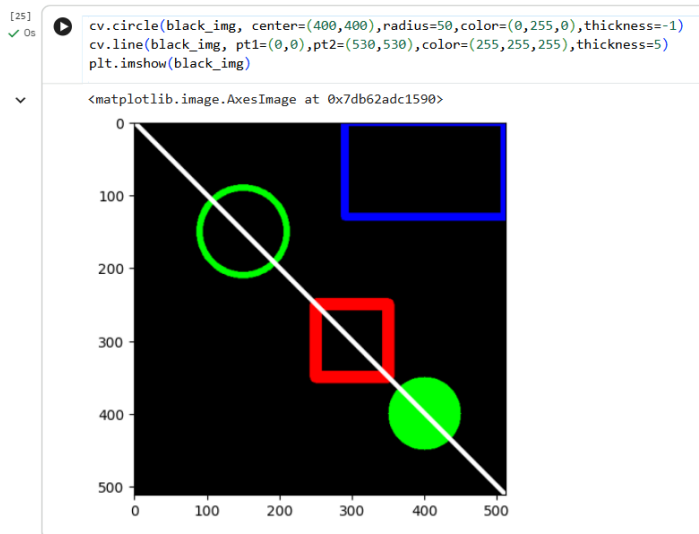
Selanjutnya ditambah menambahkan bentuk persegi sesuai koordinat pt1 dan pt2 yang tertulis pada kode program.



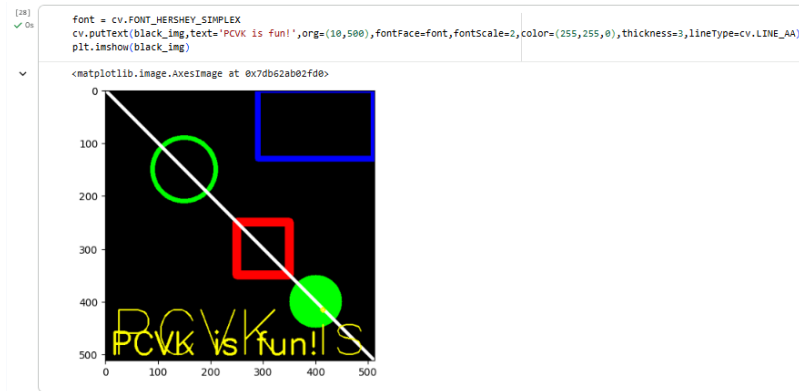
Tahap selanjutnya ditambah menambahkan bentuk lingkaran sesuai radius yang tertulis pada kode program.



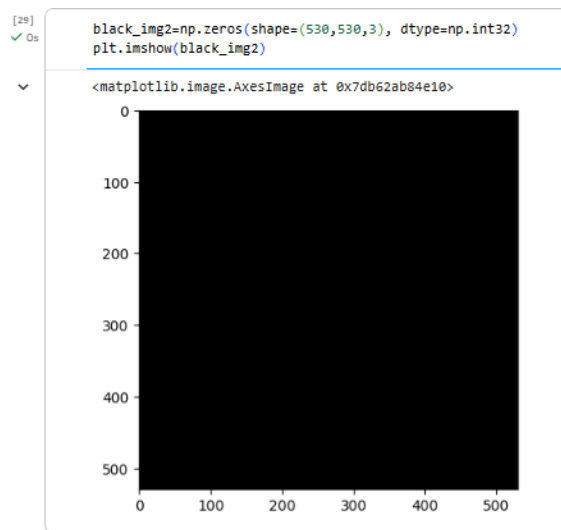
Kemudian dilakukan penambahan lingkaran berwarna dan garis sesuai koordinat pt1 dan pt2 sebagai berikut.



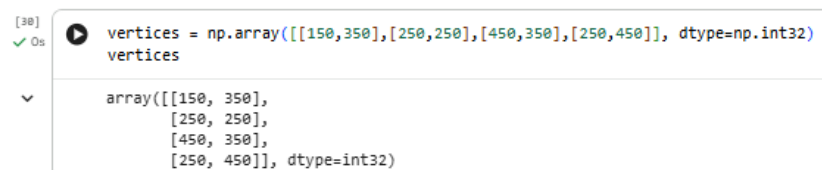
Penambahan text dengan font yang telah tertulis dengan ukuran yang sudah ditentukan.



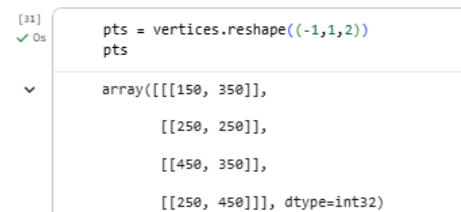
Pembuatan black image kembali dilakukan dengan tipe data int32



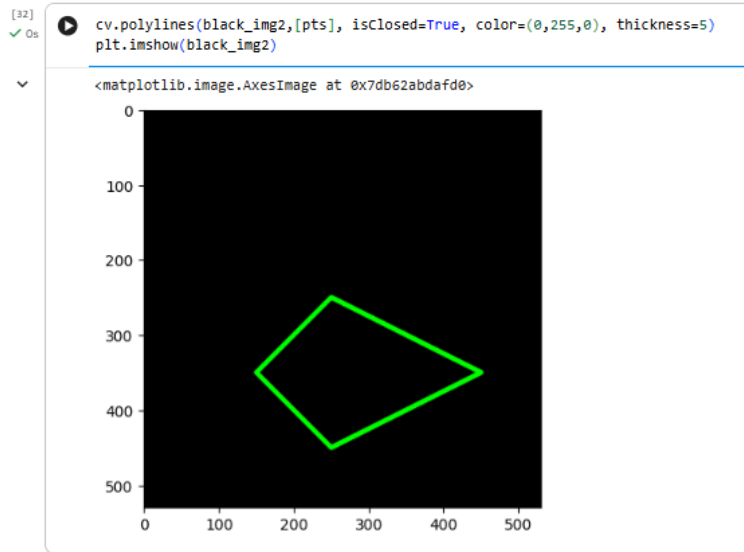
Berikut adalah kode program untuk inisialisasi NumPy array dengan tipe data int32



Array tersebut kemudian di reshape sebagai berikut



Penambahan polyline pada black image kedua yang telah dibuat.



7. Setelah semua kode selesai simpan “Week02.ipynb” pada GitHub Anda dengan memilih File kemudian “Save a copy in GitHub”.

PERTANYAAN PRAKTIKUM

Jawab pertanyaan berikut pada sel Markdown di dalam notebook Anda. Setiap jawaban yang menyebut nilai atau ukuran wajib disertai keluaran program.

1. Tampilkan satu citra bebas menggunakan `cv2.imshow()` dan `plt.imshow()` tanpa konversi warna apa pun. Mengapa hasilnya berbeda? Buktikan dengan mencetak nilai satu piksel yang sama dari kedua cara pembacaan tersebut.

Jawab :

```
[34] ✓ %s
from skimage import data
# Pertanyaan 1: Tampilkan citra dengan cv2.imshow() dan plt.imshow() tanpa konversi
# Membaca citra dengan OpenCV (format BGR)
img_bgr = data.astronaut() # skimage: format RGB
img_cv = img_bgr[:, :, ::-1].copy() # konversi ke BGR (simulasi cv2.imread)

print('=== Tampilan dengan cv2.imshow() (format BGR) ===')
cv2.imshow('img_cv')

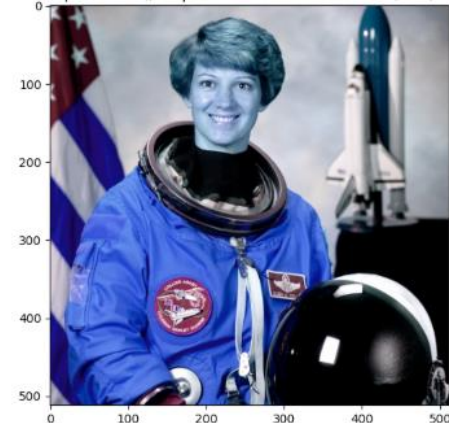
print('\n=== Tampilan dengan plt.imshow() tanpa konversi (masih BGR) ===')
plt.figure(figsize=(6, 6))
plt.imshow(img_cv) # plt mengasumsikan RGB, tapi input BGR -> warna tertukar
plt.title('plt.imshow() tanpa konversi - Warna Tertukar (BGR)')
plt.show()

# Buktikan nilai piksel
print(f'\nNilai piksel [100,100] format BGR (cv2): {img_cv[100, 100]}')
print(f'Nilai piksel [100,100] format RGB (skimage): {img_bgr[100, 100]}')
print('\nPerhatikan: urutan channel B dan R tertukar!')
print('Kesimpulan: Perbedaan tampilan disebabkan perbedaan urutan kanal BGR vs RGB.')
```

=== Tampilan dengan cv2.imshow() (format BGR) ===



=== Tampilan dengan plt.imshow() tanpa konversi (masih BGR) ===
plt.imshow() tanpa konversi - Warna Tertukar (BGR)



```
Nilai piksel [100,100] format BGR (cv2): [169 176 187]
Nilai piksel [100,100] format RGB (skimage): [187 176 169]
```

Perhatikan: urutan channel B dan R tertukar!

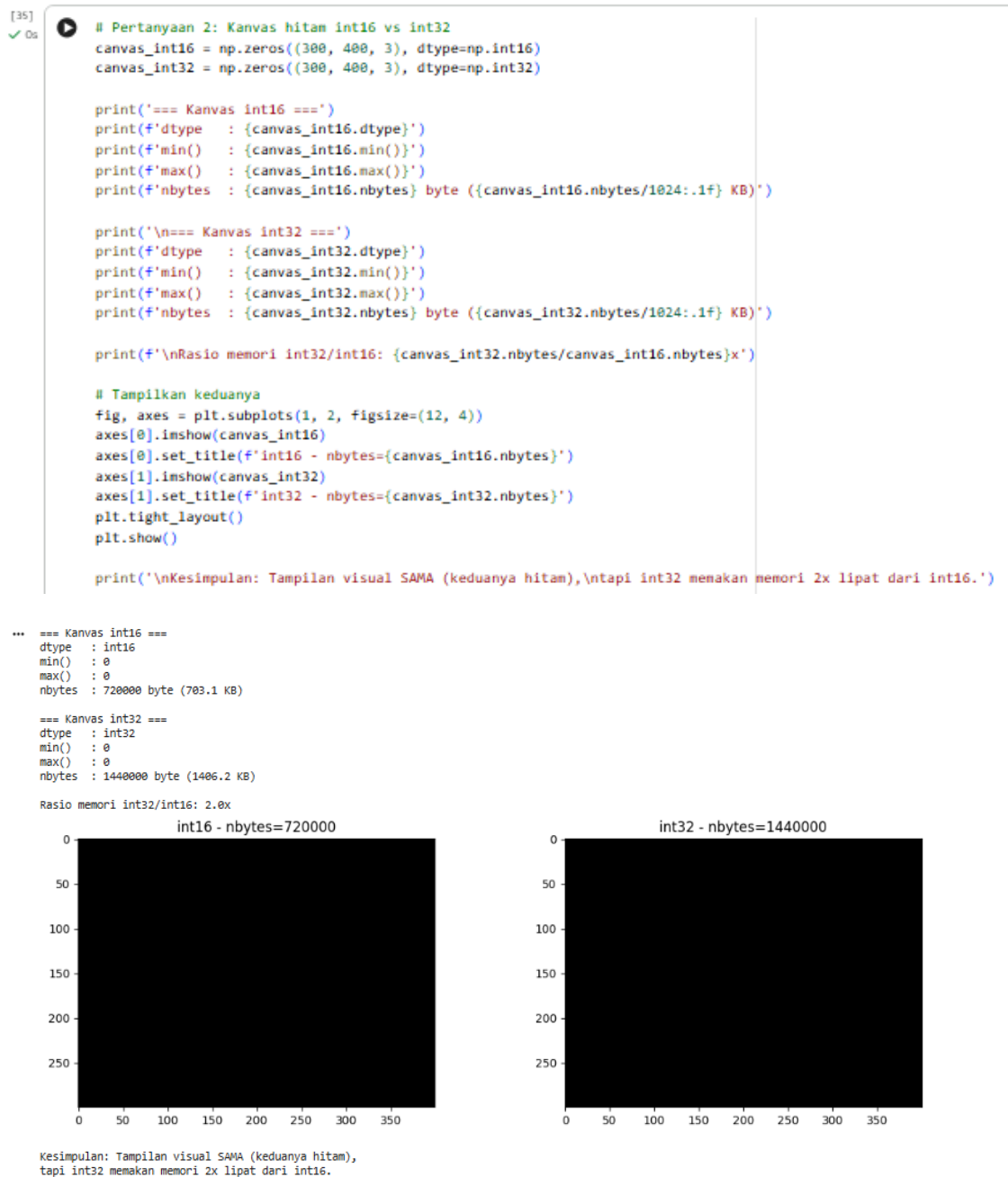
Kesimpulan: Perbedaan tampilan disebabkan perbedaan urutan kanal BGR vs RGB.

Hasilnya berbeda karena `cv2.imshow()` menampilkan citra dalam format BGR (default OpenCV), sedangkan `plt.imshow()` mengasumsikan format RGB. Ketika citra BGR ditampilkan dengan `plt.imshow()` tanpa konversi, channel

Blue dan Red tertukar sehingga warna tampak tidak natural. Bukti: nilai piksel yang sama memiliki urutan kanal yang terbalik antara kedua format.

2. Buat kanvas hitam berukuran sama dengan tipe data int16 dan int32. Bandingkan dtype, min(), max(), dan nbytes keduanya. Apakah tampilannya berbeda? Apakah ukuran memorinya berbeda? Jelaskan alasannya.

Jawab :



Tampilannya tidak berbeda (keduanya hitam karena semua piksel bernilai 0), namun ukuran memori berbeda — int32 menggunakan 2x lipat memori dibanding int16. $\text{int16} = 2 \text{ byte/eleman}$, $\text{int32} = 4 \text{ byte/eleman}$. Perbedaan terasa pada operasi aritmatika: int16 rentan overflow (batas ± 32.767), sedangkan int32 mampu menampung nilai lebih besar dan wajib digunakan untuk `cv2.polyline()`.

3. Apa kegunaan baris `from google.colab.patches import cv2_imshow`, dan mengapa `cv2.imshow()` bawaan OpenCV tidak dapat dipakai di Google Colab?

Jawab :

Baris `from google.colab.patches import cv2_imshow` digunakan untuk mengimpor fungsi `cv2_imshow()` yang merupakan pengganti `cv2.imshow()` khusus untuk lingkungan Google Colab.

`cv2.imshow()` bawaan OpenCV tidak dapat dipakai di Google Colab karena: `cv2.imshow()` membutuhkan GUI display server (jendela grafis) pada sistem operasi untuk menampilkan gambar.

4. Citra yang dibaca dengan `skimage.io.imread()` ditampilkan berdampingan dengan hasil `cv2.cvtColor(img, cv2.COLOR_BGR2RGB)` atas citra yang sama. Manakah yang menampilkan warna sebenarnya, dan manakah yang kanalnya tertukar? Buktikan dengan nilai piksel, bukan dengan pengamatan visual saja.

Jawab :

```
[36]
✓ 1s # Pertanyaan 4: skimage.io.imread() vs cv2.cvtColor(COLOR_BGR2RGB) pada citra skimage

# Baca citra dengan skimage (sudah RGB)
img_skimage = data.astronaut() # format RGB

# Terapkan cvtColor BGR2RGB pada citra yang sudah RGB (SALAH!)
img_converted = cv2.cvtColor(img_skimage, cv2.COLOR_BGR2RGB)

# Tampilkan berdampingan
fig, axes = plt.subplots(1, 2, figsize=(14, 6))
axes[0].imshow(img_skimage)
axes[0].set_title('skimage.io.imread() - Warna BENAR (RGB)')
axes[1].imshow(img_converted)
axes[1].set_title('cv2.cvtColor(BGR2RGB) pada citra skimage - Warna TERTUKAR')
plt.tight_layout()
plt.show()

# Buktikan dengan nilai piksel
print(f'Piksel [100,100] skimage (RGB asli) : {img_skimage[100, 100]}')
print(f'Piksel [100,100] setelah cvtColor : {img_converted[100, 100]}')
print('\nPerhatikan: Channel R dan B tertukar pada hasil cvtColor!')
print('\nKesimpulan: Citra dari skimage sudah RGB. Menerapkan cvtColor(BGR2RGB)')
print('justu MEMBALIK urutan yang sudah benar, menjadikan channel tertukar.')
```



Yang menampilkan warna sebenarnya adalah citra yang dibaca dengan `skimage.io.imread()`, karena `skimage` sudah membaca citra dalam format RGB yang merupakan format standar tampilan.

5. Setelah citra ditampilkan dengan `figsize` yang jauh lebih besar, apakah nilai `img.shape` ikut berubah? Jelaskan perbedaan antara mengubah ukuran citra dan mengubah ukuran tampilan.

Jawab :





Nilai `img.shape` tidak ikut berubah ketika `figsize` diperbesar. `img.shape` tetap menunjukkan resolusi asli citra (misalnya (500, 800, 3) tetap sama).

TUGAS PRAKTIKUM – Penerapan

Berdasarkan praktikum bagian 1 dan 2 kerjakan beberapa tugas berikut :

1. Tampilkan citra hanya pada kombinasi kanal Merah-Biru dan kanal Hijau-Biru saja, dengan kanal yang tidak dipakai dinolkan. Susun keduanya berdampingan dalam satu figure yang diberi judul.

Jawab :

```
[38]
✓ 1s # Tugas Penerapan 1: Tampilkan kanal Merah-Biru dan Hijau-Biru
img_rgb = data.astronaut()

# Kanal Merah-Biru (Green = 0)
img_rb = img_rgb.copy()
img_rb[:, :, 1] = 0 # Nolkan channel Green (index 1)

# Kanal Hijau-Biru (Red = 0)
img_gb = img_rgb.copy()
img_gb[:, :, 0] = 0 # Nolkan channel Red (index 0)

fig, axes = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Tugas 1: Kombinasi Kanal warna', fontsize=16, fontweight='bold')
axes[0].imshow(img_rgb)
axes[0].set_title('Citra Asli (RGB)')
axes[1].imshow(img_rb)
axes[1].set_title('Kanal Merah-Biru (Green=0)')
axes[2].imshow(img_gb)
axes[2].set_title('Kanal Hijau-Biru (Red=0)')
plt.tight_layout()
plt.show()
```



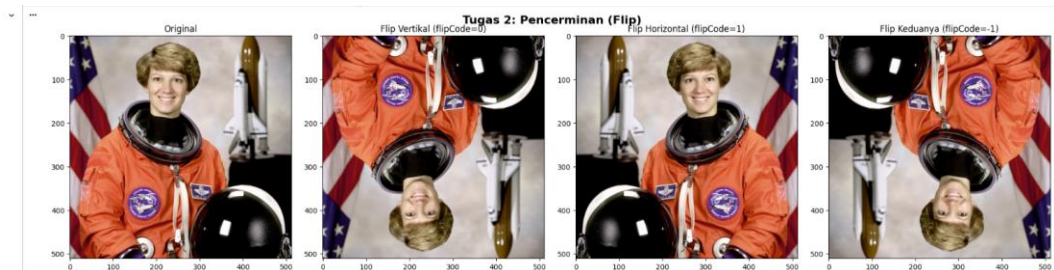

2. Tampilkan citra dalam tiga bentuk pencerminan: vertikal, horizontal, dan keduanya, dalam satu figure berisi tiga subplot yang masing-masing diberi label.

Jawab :

```
[39] # Tugas Penerapan 2: Tiga bentuk pencerminan
✓ ts img_src = data.astronaut()

img_flip_v = cv2.flip(img_src, 0) # Vertikal (atas-bawah)
img_flip_h = cv2.flip(img_src, 1) # Horizontal (kiri-kanan)
img_flip_both = cv2.flip(img_src, -1) # Keduanya

fig, axes = plt.subplots(1, 4, figsize=(20, 5))
fig.suptitle('Tugas 2: Pencerminan (Flip)', fontsize=16, fontweight='bold')
axes[0].imshow(img_src)
axes[0].set_title('Original')
axes[1].imshow(img_flip_v)
axes[1].set_title('Flip Vertikal (flipCode=0)')
axes[2].imshow(img_flip_h)
axes[2].set_title('Flip Horizontal (flipCode=1)')
axes[3].imshow(img_flip_both)
axes[3].set_title('Flip Keduanya (flipCode=-1)')
plt.tight_layout()
plt.show()
```



3. Gambarkan sebuah *rectangle* dan sebuah *circle* pada bagian wajah / badan dari foto aktivitas Anda sendiri (bukan pasfoto), lalu tambahkan teks berisi nama Anda dengan pilihan font, ukuran, dan warna yang Anda tentukan sendiri.

Jawab :

```

from google.colab import files
import cv2
import matplotlib.pyplot as plt

uploaded = files.upload()
filename = next(iter(uploaded))
img = cv2.imread(filename)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)

# Deteksi wajah
faces = face_cascade.detectMultiScale(
    gray,
    scaleFactor=1.1,
    minNeighbors=5,
    minSize=(30, 30)
)

# Gambar rectangle dan circle
img_face = img.copy()

for (x, y, w, h) in faces:

    # Rectangle hijau
    cv2.rectangle(
        img_face,
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        3
    )

    # Titik tengah wajah
    center_x = x + w // 2
    center_y = y + h // 2

    # Radius lingkaran
    radius = max(w, h) // 2

    # Circle merah
    cv2.circle(
        img_face,
        (center_x, center_y),
        radius,
        (255, 0, 0),
        3
    )

print(f"Berhasil mendeteksi {len(faces)} wajah.")

# BGR -> RGB agar warna benar saat ditampilkan matplotlib
img_face_rgb = cv2.cvtColor(img_face, cv2.COLOR_BGR2RGB)

# Tampilkan
plt.figure(figsize=(8, 8))
plt.imshow(img_face_rgb)
plt.title("Tugas D3.7: Rectangle dan Circle pada Wajah")
plt.axis("off")
plt.show()

```



TUGAS PRAKTIKUM - Analisa

1. Rumuskan satu pertanyaan tentang citra digital yang dapat Anda jawab menggunakan kode dari modul ini. Contoh pertanyaan: berapa persen piksel pada foto KTM Anda yang lebih gelap dari nilai 100? apakah mengecilkan lalu membesarkan citra mengubah rata-rata intensitasnya? Tuliskan pertanyaan Anda, rancangan program, data hasilnya, dan kesimpulan Anda.

Jawab :

[44]

```

# TUGAS ANALISA:

# === Upload gambar dari komputer ===
from google.colab import files
uploaded = files.upload()
filename = list(uploaded.keys())[0]
print(f'File yang diupload: {filename}')

# Baca gambar yang diupload
img_analisa = cv2.imread(filename)
img_analisa = cv2.cvtColor(img_analisa, cv2.COLOR_BGR2RGB)

# === ANALISA 1: Persentase piksel gelap ===
img_gray = cv2.cvtColor(img_analisa, cv2.COLOR_RGB2GRAY)

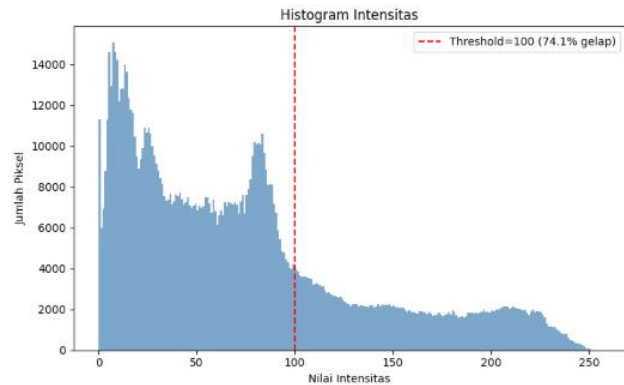
total_piksel = img_gray.size
piksel_gelap = np.sum(img_gray < 100)
persentase_gelap = (piksel_gelap / total_piksel) * 100

print('='*60)
print('ANALISA 1: Persentase Piksel Gelap (< 100)')
print('='*60)
print(f'Total piksel      : {total_piksel}')
print(f'Piksel gelap (<100) : {piksel_gelap}')
print(f'Persentase gelap    : {persentase_gelap:.2f}%')

# Visualisasi histogram
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
axes[0].imshow(img_gray, cmap='gray')
axes[0].set_title('Citra Grayscale')
axes[1].hist(img_gray.ravel(), bins=256, range=(0, 256), color='steelblue', alpha=0.7)
axes[1].axvline(x=100, color='red', linestyle='--', label=f'Threshold=100 ({persentase_gelap:.1f}% gelap)')
axes[1].set_title('Histogram Intensitas')
axes[1].set_xlabel('Nilai Intensitas')
axes[1].set_ylabel('Jumlah Piksel')
axes[1].legend()
plt.tight_layout()
plt.show()

```

Choose Files: 5.jpeg
 5.jpeg(image/jpeg) • 895240 bytes, last modified: 02/08/2026 - 100% done
 Saving 5.jpeg to 5.jpeg
 File yang diupload: 5.jpeg
 =====
 ANALISA 1: Persentase Piksel Gelap (< 100)
 =====
 Total piksel : 1142864
 Piksel gelap (<100) : 846145
 Persentase gelap : 74.09%



[45]

```

# == ANALISA 2: Apakah mengecilkan lalu membesarkan mengubah rata-rata intensitas? ==
print('*60')
print('ANALISA 2: Efek Resize (Kecikan -> Besarkan) pada Intensitas')
print('*60')

# Menggunakan gambar yang sama dari upload di Analisa 1
img_original = img_analisa.copy()
h, w = img_original.shape[:2]
rata_original = np.mean(img_original.astype(np.float64))
print(f'Ukuran asli      : {w}x{h}')
print(f'Rata-rata asli    : {rata_original:.4f}')

# Kecilkan menjadi 1/4
img_kecil = cv2.resize(img_original, (w//4, h//4), interpolation=cv2.INTER_AREA)
rata_kecil = np.mean(img_kecil.astype(np.float64))
print(f'\nUkuran kecil    : {w//4}x{h//4}')
print(f'Rata-rata kecil    : {rata_kecil:.4f}')

# Besarkan kembali ke ukuran asli
img_besar = cv2.resize(img_kecil, (w, h), interpolation=cv2.INTER_CUBIC)
rata_besar = np.mean(img_besar.astype(np.float64))
print(f'\nUkuran kembali   : {w}x{h}')
print(f'Rata-rata kembali   : {rata_besar:.4f}')

selisih = abs(rata_original - rata_besar)
print(f'\nSelisih rata-rata : {selisih:.4f}')
print(f'Perubahan        : {selisih/rata_original*100:.4f}%')

# Tampilkan perbandingan visual
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
axes[0].imshow(img_original)
axes[0].set_title(f'Original ({w}x{h})\nMean={rata_original:.2f}')
axes[1].imshow(img_kecil)
axes[1].set_title(f'Dikecilkan ({w//4}x{h//4})\nMean={rata_kecil:.2f}')
axes[2].imshow(img_besar)
axes[2].set_title(f'Dibesarkan Kembali ({w}x{h})\nMean={rata_besar:.2f}')
plt.suptitle('Analisa 2: Efek Resize pada Rata-rata Intensitas', fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()

print('\nKesimpulan: Mengecilkan lalu membesarkan citra MENGUBAH rata-rata')
print('intensitasnya (meskipun sedikit). Ini terjadi karena proses interpolasi')
print('tidak sempurna - detail yang hilang saat pengecilan tidak bisa dikembalikan.')

```

```

=====
ANALISA 2: Efek Resize (Kecikan -> Besarkan) pada Intensitas
=====
Ukuran asli      : 924x1236
Rata-rata asli    : 73.2578

Ukuran kecil     : 231x309
Rata-rata kecil   : 73.2575

Ukuran kembali   : 924x1236
Rata-rata kembali : 73.2774

Selisih rata-rata : 0.0196
Perubahan         : 0.0267%

```

Analisa 2: Efek Resize pada Rata-rata Intensitas



Kesimpulan: Mengecilkan lalu membesarkan citra MENGUBAH rata-rata intensitasnya (meskipun sedikit). Ini terjadi karena proses interpolasi tidak sempurna - detail yang hilang saat pengecilan tidak bisa dikembalikan.

TUGAS PRAKTIKUM - Aplikasi

Pada bagian ini, Anda hanya diperbolehkan memakai fungsi yang telah dipelajari pada modul ini: pembacaan berkas, pengirisan, `cv2.resize`, `cv2.flip`, `cv2.cvtColor`, fungsi penggambaran OpenCV, serta pembuatan kanvas dengan NumPy. Selain itu, program harus bersifat adaptif, dimana setiap ukuran, posisi, atau ketebalan harus dihitung dari ukuran citra masukan, bukan ditulis sebagai angka tetap pada kode program.

Studi Kasus 1: Panitia sebuah kegiatan perlu membubuhkan label identitas pada ratusan foto dokumentasi. Foto berasal dari beberapa kamera dan ponsel, sehingga ukurannya bermacam-macam. Label harus terlihat seimbang pada semua foto tanpa disetel satu per satu. Fungsi yang sama dikenakan pada dua citra berbeda ukuran. Tinggi bar dan ukuran huruf ikut menyesuaikan, sehingga proporsinya tetap sama.

Jawaban :

```
[40]
✓ 32s # STUDI KASUS 1: Label identitas adaptif pada foto berbagai ukuran
# Label harus proporsional terhadap ukuran citra masukan

def tambah_label_identitas(img, keterangan, kegiatan):
    """Menambahkan label identitas adaptif pada citra.
    Semua ukuran dihitung dari dimensi citra, bukan angka tetap."""
    result = img.copy()
    h, w = result.shape[:2]

    # Tinggi bar = 12% dari tinggi citra
    bar_height = int(h * 0.12)

    # Buat overlay semi-transparan untuk bar
    overlay = result.copy()
    cv2.rectangle(overlay, (0, h - bar_height), (w, h), (0, 0, 0), -1)
    result = cv2.addWeighted(overlay, 0.6, result, 0.4, 0)

    # Ukuran font proporsional terhadap lebar citra
    font_scale = w / 800 # basis: lebar 800px -> fontScale 1.0
    thickness = max(1, int(font_scale * 2))

    # Teks baris 1: Keterangan
    y1 = h - bar_height + int(bar_height * 0.40)
    cv2.putText(result, keterangan, (int(w * 0.02), y1),
                cv2.FONT_HERSHEY_SIMPLEX, font_scale * 0.7,
                (255, 255, 255), thickness, cv2.LINE_AA)

    # Teks baris 2: Kegiatan
    y2 = h - bar_height + int(bar_height * 0.80)
    cv2.putText(result, kegiatan, (int(w * 0.02), y2),
                cv2.FONT_HERSHEY_SIMPLEX, font_scale * 0.5,
                (200, 200, 200), max(1, thickness - 1), cv2.LINE_AA)

    return result

# === Upload 2 foto dari komputer (portrait dan landscape) ===
from google.colab import files

print('Upload foto PORTRAIT:')
uploaded1 = files.upload()
file1 = list(uploaded1.keys())[0]
img1 = cv2.imread(file1)
img1 = cv2.cvtColor(img1, cv2.COLOR_BGR2RGB)
print(f'Foto portrait: {file1} ({img1.shape[1]}x{img1.shape[0]})')

print('\nUpload foto LANDSCAPE:')
uploaded2 = files.upload()
file2 = list(uploaded2.keys())[0]
img2 = cv2.imread(file2)
img2 = cv2.cvtColor(img2, cv2.COLOR_BGR2RGB)
print(f'Foto landscape: {file2} ({img2.shape[1]}x{img2.shape[0]})')

# Terapkan label identitas pada kedua foto
result1 = tambah_label_identitas(img1, 'Juara Lomba Transporter EMC 2025', 'Praktikum PVCK')
result2 = tambah_label_identitas(img2, 'Acara Pemilihan Ketua Umum 2026/2027', 'Praktikum PVCK')

fig, axes = plt.subplots(1, 2, figsize=(16, 6))
fig.suptitle('Studi Kasus 1: Label Identitas Adaptif', fontsize=16, fontweight='bold')
axes[0].imshow(result1)
axes[0].set_title(f'Foto Portrait ({img1.shape[1]}x{img1.shape[0]})')
axes[1].imshow(result2)
axes[1].set_title(f'Foto Landscape ({img2.shape[1]}x{img2.shape[0]})')
plt.tight_layout()
plt.show()

print('Label proporsional pada kedua foto meskipun ukurannya berbeda.')
```

Upload foto PORTRAIT:
 [Choose file] 2.jpeg
 2.jpeg(image/peg) - 996243 bytes, last modified: 10/08/2028 - 100% done
 Saving 2.jpeg to 2 (2).jpeg
 Foto portrait: 2 (2).jpeg (2268x4032)

Upload foto LANDSCAPE:
 [Choose file] 8.jpeg
 8.jpeg(image/peg) - 4814602 bytes, last modified: 10/08/2028 - 100% done
 Saving 8.jpeg to 8 (1).jpeg
 Foto landscape: 8 (1).jpeg (4320x2880)

Studi Kasus 1: Label Identitas Adaptif



Label proporsional pada kedua foto meskipun ukurannya berbeda.

Studi Kasus 2: Marketplace seperti Tokopedia dan Shopee menampilkan foto produk dalam bingkai persegi 1:1. Foto penjual yang berukuran potrait atau lanskap akan terpotong atau terlihat tidak proporsional bila diunggah begitu saja. Bangun program yang mengubah foto berukuran apa pun menjadi thumbnail persegi tanpa mengubah rasio aslinya, dengan sisa ruang diisi kanvas berwarna. Dalam hal ini, rasio asli tetap terjaga, citra terpasang tepat di tengah kanvas, tambahkan logo nama toko berukuran proporsional pada gambar.

Jawab :

```
[51]
✓ 40s # STUDI KASUS 2: Thumbnail persegi 1:1 untuk marketplace
# Tanpa mengubah rasio asli, sisa ruang diisi kanvas berwarna + logo toko

def buat_thumbnail_persegi(img, nama_toko, bg_color=(240, 240, 240)):
    """Mengubah foto menjadi thumbnail persegi 1:1 tanpa mengubah rasio asli.
    Semua ukuran adaptif terhadap dimensi citra."""
    h, w = img.shape[:2]

    # Ukuran kanvas persegi = sisi terpanjang
    sisi = max(h, w)

    # Buat kanvas persegi dengan warna background
    canvas = np.full((sisi, sisi, 3), bg_color, dtype=np.uint8)

    # Hitung posisi agar citra di tengah
    y_offset = (sisi - h) // 2
    x_offset = (sisi - w) // 2

    # Tempatkan citra di tengah kanvas
    canvas[y_offset:y_offset + h, x_offset:x_offset + w] = img

    # Tambahkan logo nama toko (proporsional)
    font_scale = sisi / 600 # basis: 600px -> fontScale 1.0
    thickness = max(1, int(font_scale * 2))

    # Hitung ukuran teks untuk positioning
    text_size = cv2.getTextSize(nama_toko, cv2.FONT_HERSHEY_SIMPLEX,
                                font_scale * 0.6, thickness)[0]

    # Background kotak untuk teks (pojok kanan bawah)
    padding = int(sisi * 0.015)
    text_x = sisi - text_size[0] - padding * 3
    text_y = sisi - padding * 3

    # Rectangle semi-transparan di belakang teks
    overlay = canvas.copy()
    cv2.rectangle(overlay,
                  (text_x - padding, text_y - text_size[1] - padding),
                  (sisi - padding, text_y + padding),
                  (50, 50, 50), -1)
    canvas = cv2.addWeighted(overlay, 0.7, canvas, 0.3, 0)

    # Teks nama toko
    cv2.putText(canvas, nama_toko, (text_x, text_y),
                cv2.FONT_HERSHEY_SIMPLEX, font_scale * 0.6,
                (255, 255, 255), thickness, cv2.LINE_AA)

    return canvas
```



```
# === Upload 2 foto dari komputer (portrait dan landscape) ===
from google.colab import files

print('Upload foto PORTRAIT:')
uploaded1 = files.upload()
file1 = list(uploaded1.keys())[0]
img_portrait = cv2.imread(file1)
img_portrait = cv2.cvtColor(img_portrait, cv2.COLOR_BGR2RGB)
print(f'Foto portrait: {file1} ({img_portrait.shape[1]}x{img_portrait.shape[0]})')

print('\nUpload foto LANDSCAPE:')
uploaded2 = files.upload()
file2 = list(uploaded2.keys())[0]
img_landscape = cv2.imread(file2)
img_landscape = cv2.cvtColor(img_landscape, cv2.COLOR_BGR2RGB)
print(f'Foto landscape: {file2} ({img_landscape.shape[1]}x{img_landscape.shape[0]})')

thumb1 = buat_thumbnail_persegi(img_portrait, 'TokoNISHO', bg_color=(245, 245, 245))
thumb2 = buat_thumbnail_persegi(img_landscape, 'TokoNISHO', bg_color=(245, 245, 245))

fig, axes = plt.subplots(2, 2, figsize=(14, 14))
fig.suptitle('Studi Kasus 2: Thumbnail Persegi 1:1 untuk Marketplace',
             fontsize=16, fontweight='bold')

axes[0][0].imshow(img_portrait)
axes[0][0].set_title(f'Foto Portrait Asli ({img_portrait.shape[1]}x{img_portrait.shape[0]})')

axes[0][1].imshow(thumb1)
axes[0][1].set_title(f'Thumbnail 1:1 ({thumb1.shape[1]}x{thumb1.shape[0]})')

axes[1][0].imshow(img_landscape)
axes[1][0].set_title(f'Foto Landscape Asli ({img_landscape.shape[1]}x{img_landscape.shape[0]})')

axes[1][1].imshow(thumb2)
axes[1][1].set_title(f'Thumbnail 1:1 ({thumb2.shape[1]}x{thumb2.shape[0]})')

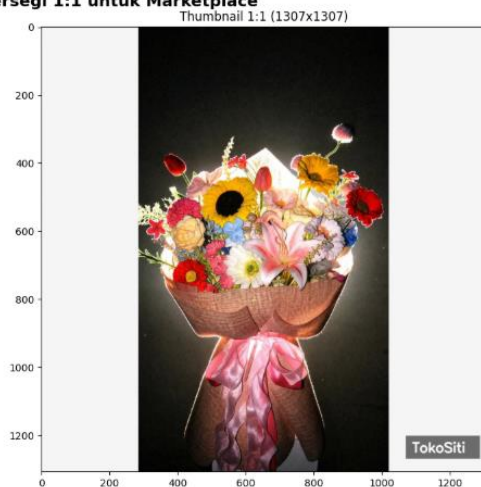
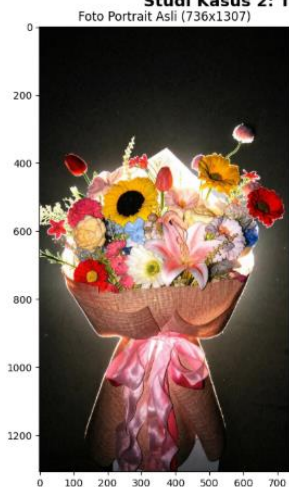
plt.tight_layout()
plt.show()

print('Kedua foto (portrait & landscape) berhasil diubah menjadi persegi 1:1')
print('tanpa mengubah rasio asli, dengan logo toko proporsional.')
```

Upload foto PORTRAIT:
 buket potrait.jpg
 buket potrait.jpg (image/jpeg) - 175148 bytes, last modified: 01/09/2026 - 100% done
 Saving buket potrait.jpg to buket potrait.jpg
 Foto portrait: buket potrait.jpg (736x1307)

Upload foto LANDSCAPE:
 buket landscape.jpg
 buket landscape.jpg (image/jpeg) - 62670 bytes, last modified: 01/09/2026 - 100% done
 Saving buket landscape.jpg to buket landscape.jpg
 Foto landscape: buket landscape.jpg (736x414)

Studi Kasus 2: Thumbnail Persegi 1:1 untuk Marketplace





Kedua foto (portrait & landscape) berhasil diubah menjadi persegi 1:1 tanpa mengubah rasio asli, dengan logo toko proporsional.

